

lab_weather_partial

November 30, 2025

1 Lab: Fine-tuning an Image Model for Weather Classification

As discussed in class, training large computer vision models from scratch takes enormous amounts of data and computational training. If you don't have days to wait, it is best to **fine tune** a pre-existing well-trained model. In this lab, we will demonstrate how to fine tune a model for weather classification.

In going through this lab, you will learn to:

- Download a dataset from **Kagglehub**
- Use operating system python calls to explore files and perform tasks such as splitting the data into training and test
- Download a **base model**. In this lab, we use the simple MobileNetV2 since it is computationally easy
- **Fine tune** the model with a simple **classifier head**
- Add progress bars in trianing and evalating while training is occuring

1.1 Using Google Colab's Free Tier GPU

The lab is greatly assisted with a GPU. While most laptops have GPUs, they are generally not useful for ML. So, assuming you do not have access to a specific machine designed for ML, I suggest you use Google colab. You can follow the instructions on this [Stanford blog](#) to select a GPU.

The free tier of Google Colab comes with a T4 Telsa GPU which is sufficient for this lab. For more money, you can select the A100 GPU which a bit faster. But, for this relatively small model, there is not much gain going to the A100 GPU (I found it only goes about 50% faster).

1.2 Loading the dataset from Kaggle

We will use a [Kaggle dataset](#) with images of scenes in different weather conditions like frost, rain, ...

First, we download the data set with the following command. The total dataset is almost 1 GB large, so the downloading could take a few minutes.

```
[3]: import kagglehub
import os
import shutil

# Determine target directory within unit10_cnn
```

```

base_dir = os.path.dirname(os.path.abspath(__file__)) if "__file__" in_
↪globals() else os.getcwd()
target_dir = os.path.join(base_dir, "dataset")

# Download dataset via kagglehub (downloads to cache directory by default)
cached_path = kagglehub.dataset_download("jehanbhatena/weather-dataset")

# Move (or copy) dataset into our desired directory under unit10_cnn
if not os.path.exists(target_dir):
    shutil.copytree(cached_path, target_dir)
else:
    print("Target dataset directory already exists.")

path = target_dir
print("Path to dataset files:", path)

```

Target dataset directory already exists.

Path to dataset files: /Users/willem/Library/CloudStorage/GoogleDrive-wdn2012@nyu.edu/My Drive/F2025/introml/unit10_cnn/dataset

After the download, there will be one sub-folder for each category. The directory structure will look like:

```

dataset_path/
  dew/
    image1.jpg
    image2.jpg
    ...
  fogsmog/
    image1.jpg
    image2.jpg
    ...
  frost/
    image1.jpg
    image2.jpg
    ...
  glaze/
    image1.jpg
    image2.jpg
    ...

```

Print the names of the sub-folders and the number of files in each sub-folder. Also, create a list `categories` with Some useful commands are: * `os.listdir` which lists all the directories * `os.path.join(dataset_path, d)`: creates the path for the subfolder `d` in `dataset_path`.

```

[ ]: import os
import pandas as pd
dataset_path = os.path.join(path, "dataset", "dataset")

```

```
# TODO: Populate the lists below with category names and their corresponding
↪file counts
categories = os.listdir(dataset_path)
file_counts = [len(os.listdir(os.path.join(dataset_path, d))) for d in
↪categories]

# TODO: Create a DataFrame from the lists and print it
df = pd.DataFrame({'category': categories, 'file_count': file_counts})
print(df)
```

	category	file_count
0	lightning	377
1	sandstorm	692
2	glaze	639
3	rain	526
4	rime	1160
5	frost	475
6	fogsmog	851
7	hail	591
8	dew	698
9	rainbow	232
10	snow	621

Next, randomly select an image from each category and print it.

- Use the `fig, axes = plt.subplot(...)` to create an array to plot the images.
- Loop over the categories
- In each category directory, randomly select an image file
- Use the `img = Image.open(img_path)` to load the image.
- Use `ax[i].imshow(img)` to display the image.

```
[5]: import matplotlib.pyplot as plt
import numpy as np
import random
from PIL import Image
import os

nclass = len(categories)
nrow = 2
ncol = int(np.ceil(nclass / nrow))

fig, axes = plt.subplots(nrow, ncol, figsize=(15, 5))
axes = axes.flatten()

for i, category in enumerate(categories):
```

```

category_path = os.path.join(dataset_path, category)
if os.path.isdir(category_path):
    images = [f for f in os.listdir(category_path) if f.lower().endswith(('.png', '.jpg', '.jpeg'))]
    if images:
        random_image = random.choice(images)
        img_path = os.path.join(category_path, random_image)

        img = Image.open(img_path)
        axs[i].imshow(img)
        axs[i].set_title(category)
        axs[i].axis('off')

for j in range(i + 1, len(axs)):
    axs[j].axis('off')

plt.tight_layout()
plt.show()

```



1.3 Creating a Training and Test Datasets

This particular dataset has a single set of images. Write code that will create two directories **train** and **test**, each with a sub-folder structure with one set of images per class. You should randomly place a fraction **split_ratio** of the images in the **train** folder, and the remaining in the **test** folder.

```

[9]: train_dir = os.path.normpath(os.path.join(path, '..', "train"))
test_dir = os.path.normpath(os.path.join(path, '..', "test"))

split_ratio = 0.8

for category in categories:
    category_path = os.path.join(dataset_path, category)

```

```

    images = [f for f in os.listdir(category_path) if f.lower().endswith(('.'
    ↪png', '.jpg', '.jpeg'))]
    random.shuffle(images)

    split_index = int(len(images) * split_ratio)
    train_images = images[:split_index]
    test_images = images[split_index:]

    train_path = os.path.join(train_dir, category)
    test_path = os.path.join(test_dir, category)

    os.makedirs(train_path, exist_ok=True)
    os.makedirs(test_path, exist_ok=True)

    for img in train_images:
        src = os.path.join(category_path, img)
        dst = os.path.join(train_path, img)
        shutil.copy2(src, dst)

    for img in test_images:
        src = os.path.join(category_path, img)
        dst = os.path.join(test_path, img)
        shutil.copy2(src, dst)

train_counts = []
test_counts = []

for category in categories:
    train_count = len(os.listdir(os.path.join(train_dir, category)))
    test_count = len(os.listdir(os.path.join(test_dir, category)))
    train_counts.append(train_count)
    test_counts.append(test_count)
    print(f"{category}: {train_count} training, {test_count} test")

df = pd.DataFrame({
    'category': categories,
    'train_count': train_counts,
    'test_count': test_counts
})
print(df)

```

```

lightning: 361 training, 136 test
sandstorm: 668 training, 254 test
glaze: 611 training, 228 test
rain: 501 training, 187 test
rime: 1108 training, 412 test
frost: 447 training, 162 test
fogsmog: 820 training, 311 test

```

```

hail: 564 training, 211 test
dew: 677 training, 259 test
rainbow: 223 training, 85 test
snow: 599 training, 228 test

```

	category	train_count	test_count
0	lightning	361	136
1	sandstorm	668	254
2	glaze	611	228
3	rain	501	187
4	rime	1108	412
5	frost	447	162
6	fogsmog	820	311
7	hail	564	211
8	dew	677	259
9	rainbow	223	85
10	snow	599	228

1.4 Downloading the base model

Pytorch has a number of excellent pre-trained models that we can use for fine tuning. For this lab, to make the training easy, we will use a lightweight model called **MobileNetV2**. MobileNetV2 is a CNN developed by Google that targeting mobile devices that are computationally limited. It uses an architecture with inverted residual blocks and linear bottlenecks to improve performance while keeping computational costs low. A nice summary of the model can be found in this [Medium post](#). We can download the model as follows.

First we download the key packages and set a `transform` needed for the MobileNetV2.

```

[10]: import os
import torch
import torchvision
import torchvision.transforms as transforms
from torchvision import datasets, models
from torch.utils.data import DataLoader
import torch.nn as nn
import torch.optim as optim

[11]: transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])

```

Next, we create `DataLoader` classes to load the models.

```

[12]: # TODO: Create the training and test datasets. Use the `datasets.ImageFolder`
      ↪class
      # with the appropriate directory and transform.

```

```

train_dataset = datasets.ImageFolder(train_dir, transform=transform)
test_dataset = datasets.ImageFolder(test_dir, transform=transform)

# TODO: Create the training and test dataloaders. Use a batch size of 16.
train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=16, shuffle=False)

# TODO: Print the number of classes.
print(f"Number of classes: {len(categories)}")

```

Number of classes: 11

Now we download the pre-trained model.

```

[13]: model = models.mobilenet_v2(pretrained=True)

/Users/willem/Library/CloudStorage/GoogleDrive-wdn2012@nyu.edu/My
Drive/F2025/introml/introml/lib/python3.12/site-
packages/torchvision/models/_utils.py:208: UserWarning: The parameter
'pretrained' is deprecated since 0.13 and may be removed in the future, please
use 'weights' instead.
  warnings.warn(
/Users/willem/Library/CloudStorage/GoogleDrive-wdn2012@nyu.edu/My
Drive/F2025/introml/introml/lib/python3.12/site-
packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a
weight enum or `None` for 'weights' are deprecated since 0.13 and may be removed
in the future. The current behavior is equivalent to passing
`weights=MobileNet_V2_Weights.IMAGENET1K_V1`. You can also use
`weights=MobileNet_V2_Weights.DEFAULT` to get the most up-to-date weights.
  warnings.warn(msg)

Downloading: "https://download.pytorch.org/models/mobilenet_v2-b0353104.pth" to
/Users/willem/.cache/torch/hub/checkpoints/mobilenet_v2-b0353104.pth
100%|          | 13.6M/13.6M [00:00<00:00, 46.5MB/s]

```

The MobileNetV2 model has two main components: - **model.features** This is the **base model**, also known as the **backbone**. It contains the convolutional layers that extract features from the input image. In transfer learning, we typically freeze this part to retain the pretrained feature extractor. - **model.classifier** This is the **classifier head**. It takes the output from **model.features** and maps it to the final class predictions. We **replace and train** this part to adapt the model to our specific task (e.g., weather classification)

First, we can see all the layers with the following command.

```

[14]: model.features

```

```

[14]: Sequential(
  (0): Conv2dNormActivation(
    (0): Conv2d(3, 32, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1),
    bias=False)
    (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True,
    track_running_stats=True)
    (2): ReLU6(inplace=True)
  )
  (1): InvertedResidual(
    (conv): Sequential(
      (0): Conv2dNormActivation(
        (0): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
        groups=32, bias=False)
        (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True,
        track_running_stats=True)
        (2): ReLU6(inplace=True)
      )
      (1): Conv2d(32, 16, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (2): BatchNorm2d(16, eps=1e-05, momentum=0.1, affine=True,
      track_running_stats=True)
    )
  )
  (2): InvertedResidual(
    (conv): Sequential(
      (0): Conv2dNormActivation(
        (0): Conv2d(16, 96, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (1): BatchNorm2d(96, eps=1e-05, momentum=0.1, affine=True,
        track_running_stats=True)
        (2): ReLU6(inplace=True)
      )
      (1): Conv2dNormActivation(
        (0): Conv2d(96, 96, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1),
        groups=96, bias=False)
        (1): BatchNorm2d(96, eps=1e-05, momentum=0.1, affine=True,
        track_running_stats=True)
        (2): ReLU6(inplace=True)
      )
      (2): Conv2d(96, 24, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (3): BatchNorm2d(24, eps=1e-05, momentum=0.1, affine=True,
      track_running_stats=True)
    )
  )
  (3): InvertedResidual(
    (conv): Sequential(
      (0): Conv2dNormActivation(
        (0): Conv2d(24, 144, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (1): BatchNorm2d(144, eps=1e-05, momentum=0.1, affine=True,

```



```

track_running_stats=True)
    (2): ReLU6(inplace=True)
    )
    (1): Conv2dNormActivation(
      (0): Conv2d(144, 144, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
groups=144, bias=False)
      (1): BatchNorm2d(144, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (2): ReLU6(inplace=True)
    )
    (2): Conv2d(144, 24, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (3): BatchNorm2d(24, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  )
)
(4): InvertedResidual(
  (conv): Sequential(
    (0): Conv2dNormActivation(
      (0): Conv2d(24, 144, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(144, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (2): ReLU6(inplace=True)
    )
    (1): Conv2dNormActivation(
      (0): Conv2d(144, 144, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1),
groups=144, bias=False)
      (1): BatchNorm2d(144, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (2): ReLU6(inplace=True)
    )
    (2): Conv2d(144, 32, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (3): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  )
)
(5): InvertedResidual(
  (conv): Sequential(
    (0): Conv2dNormActivation(
      (0): Conv2d(32, 192, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(192, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (2): ReLU6(inplace=True)
    )
    (1): Conv2dNormActivation(
      (0): Conv2d(192, 192, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
groups=192, bias=False)
      (1): BatchNorm2d(192, eps=1e-05, momentum=0.1, affine=True,

```

```

track_running_stats=True)
    (2): ReLU6(inplace=True)
    )
    (2): Conv2d(192, 32, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (3): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
    )
    (6): InvertedResidual(
      (conv): Sequential(
        (0): Conv2dNormActivation(
          (0): Conv2d(32, 192, kernel_size=(1, 1), stride=(1, 1), bias=False)
          (1): BatchNorm2d(192, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
          (2): ReLU6(inplace=True)
        )
        (1): Conv2dNormActivation(
          (0): Conv2d(192, 192, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
groups=192, bias=False)
          (1): BatchNorm2d(192, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
          (2): ReLU6(inplace=True)
        )
        (2): Conv2d(192, 32, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (3): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      )
    )
    (7): InvertedResidual(
      (conv): Sequential(
        (0): Conv2dNormActivation(
          (0): Conv2d(32, 192, kernel_size=(1, 1), stride=(1, 1), bias=False)
          (1): BatchNorm2d(192, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
          (2): ReLU6(inplace=True)
        )
        (1): Conv2dNormActivation(
          (0): Conv2d(192, 192, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1),
groups=192, bias=False)
          (1): BatchNorm2d(192, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
          (2): ReLU6(inplace=True)
        )
        (2): Conv2d(192, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (3): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      )
    )

```

```

)
(8): InvertedResidual(
  (conv): Sequential(
    (0): Conv2dNormActivation(
      (0): Conv2d(64, 384, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(384, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (2): ReLU6(inplace=True)
  )
  (1): Conv2dNormActivation(
    (0): Conv2d(384, 384, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
groups=384, bias=False)
    (1): BatchNorm2d(384, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (2): ReLU6(inplace=True)
  )
  (2): Conv2d(384, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (3): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
)
)
(9): InvertedResidual(
  (conv): Sequential(
    (0): Conv2dNormActivation(
      (0): Conv2d(64, 384, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(384, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (2): ReLU6(inplace=True)
  )
  (1): Conv2dNormActivation(
    (0): Conv2d(384, 384, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
groups=384, bias=False)
    (1): BatchNorm2d(384, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (2): ReLU6(inplace=True)
  )
  (2): Conv2d(384, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (3): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
)
)
(10): InvertedResidual(
  (conv): Sequential(
    (0): Conv2dNormActivation(
      (0): Conv2d(64, 384, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(384, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)

```

```

        (2): ReLU6(inplace=True)
    )
    (1): Conv2dNormActivation(
      (0): Conv2d(384, 384, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
groups=384, bias=False)
      (1): BatchNorm2d(384, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (2): ReLU6(inplace=True)
    )
    (2): Conv2d(384, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (3): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  )
)
(11): InvertedResidual(
  (conv): Sequential(
    (0): Conv2dNormActivation(
      (0): Conv2d(64, 384, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(384, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (2): ReLU6(inplace=True)
    )
    (1): Conv2dNormActivation(
      (0): Conv2d(384, 384, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
groups=384, bias=False)
      (1): BatchNorm2d(384, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (2): ReLU6(inplace=True)
    )
    (2): Conv2d(384, 96, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (3): BatchNorm2d(96, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  )
)
(12): InvertedResidual(
  (conv): Sequential(
    (0): Conv2dNormActivation(
      (0): Conv2d(96, 576, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(576, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (2): ReLU6(inplace=True)
    )
    (1): Conv2dNormActivation(
      (0): Conv2d(576, 576, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
groups=576, bias=False)
      (1): BatchNorm2d(576, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)

```

```

        (2): ReLU6(inplace=True)
    )
    (2): Conv2d(576, 96, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (3): BatchNorm2d(96, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
    )
    (13): InvertedResidual(
    (conv): Sequential(
    (0): Conv2dNormActivation(
    (0): Conv2d(96, 576, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (1): BatchNorm2d(576, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (2): ReLU6(inplace=True)
    )
    (1): Conv2dNormActivation(
    (0): Conv2d(576, 576, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
groups=576, bias=False)
    (1): BatchNorm2d(576, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (2): ReLU6(inplace=True)
    )
    (2): Conv2d(576, 96, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (3): BatchNorm2d(96, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
    )
    (14): InvertedResidual(
    (conv): Sequential(
    (0): Conv2dNormActivation(
    (0): Conv2d(96, 576, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (1): BatchNorm2d(576, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (2): ReLU6(inplace=True)
    )
    (1): Conv2dNormActivation(
    (0): Conv2d(576, 576, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1),
groups=576, bias=False)
    (1): BatchNorm2d(576, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (2): ReLU6(inplace=True)
    )
    (2): Conv2d(576, 160, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (3): BatchNorm2d(160, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
    )

```

```

(15): InvertedResidual(
  (conv): Sequential(
    (0): Conv2dNormActivation(
      (0): Conv2d(160, 960, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(960, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (2): ReLU6(inplace=True)
    )
    (1): Conv2dNormActivation(
      (0): Conv2d(960, 960, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
groups=960, bias=False)
      (1): BatchNorm2d(960, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (2): ReLU6(inplace=True)
    )
    (2): Conv2d(960, 160, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (3): BatchNorm2d(160, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  )
)
(16): InvertedResidual(
  (conv): Sequential(
    (0): Conv2dNormActivation(
      (0): Conv2d(160, 960, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(960, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (2): ReLU6(inplace=True)
    )
    (1): Conv2dNormActivation(
      (0): Conv2d(960, 960, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
groups=960, bias=False)
      (1): BatchNorm2d(960, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (2): ReLU6(inplace=True)
    )
    (2): Conv2d(960, 160, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (3): BatchNorm2d(160, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  )
)
(17): InvertedResidual(
  (conv): Sequential(
    (0): Conv2dNormActivation(
      (0): Conv2d(160, 960, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (1): BatchNorm2d(960, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (2): ReLU6(inplace=True)

```

```

    )
    (1): Conv2dNormActivation(
      (0): Conv2d(960, 960, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
groups=960, bias=False)
      (1): BatchNorm2d(960, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (2): ReLU6(inplace=True)
    )
    (2): Conv2d(960, 320, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (3): BatchNorm2d(320, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  )
)
(18): Conv2dNormActivation(
  (0): Conv2d(320, 1280, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (1): BatchNorm2d(1280, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (2): ReLU6(inplace=True)
)
)

```

To get some insight into the model:

- Loop over the layers in the model with `name, layer` in `model.features.named_children`
- For each layer, get the layer type with `layer_type = layer.__class__.__name__`
- Get the total number of parameters in the layers
 - You can loop over the parameters with `for p in layer.parameters`
 - Then count the number of paramters with `p.numel()`
- Print a `pandas.DataFrame` with the layer name, `layer_type`, and number of elements.
- Also, print the total number of paramters.

You should see that the model has about 2.2M parameters.

```

[15]: # TODO: Get the layer names, types, and number of parameters in each layer of
# model.features
layer_data = []
for name, layer in model.features.named_children():
    layer_datai = {'name': name, 'layer_type': layer.__class__.__name__,
↪ 'num_param': sum(p.numel() for p in layer.parameters())}
    layer_data.append(layer_datai)

# TODO: Create a pandas DataFrame and print the layer data
df = pd.DataFrame(layer_data)
print(df)

# TODO: Print the total number of parameters

```

```
print(f"Total number of parameters: {sum(layer['num_param'] for layer in layer_data)}")
```

	name	layer_type	num_param
0	0	Conv2dNormActivation	928
1	1	InvertedResidual	896
2	2	InvertedResidual	5136
3	3	InvertedResidual	8832
4	4	InvertedResidual	10000
5	5	InvertedResidual	14848
6	6	InvertedResidual	14848
7	7	InvertedResidual	21056
8	8	InvertedResidual	54272
9	9	InvertedResidual	54272
10	10	InvertedResidual	54272
11	11	InvertedResidual	66624
12	12	InvertedResidual	118272
13	13	InvertedResidual	118272
14	14	InvertedResidual	155264
15	15	InvertedResidual	320000
16	16	InvertedResidual	320000
17	17	InvertedResidual	473920
18	18	Conv2dNormActivation	412160

Total number of parameters: 2223872

We will next the number of features from the final layer of the features model. This value will be the number of inputs to the classifier head.

```
[16]: # TODO: Get the number of outputs of the final layer of model.features
num_features = model.features[-1].out_channels
print(f"Number of features: {num_features}")
```

Number of features: 1280

Now, let's look at the classifier. This is a simple model with two layers: * A dropout layer * A fully connected layer with 1000 output features for a 1000-way softmax (recall the original ImageNet has 1000 classes).

```
[17]: model.classifier
```

```
[17]: Sequential(
  (0): Dropout(p=0.2, inplace=False)
  (1): Linear(in_features=1280, out_features=1000, bias=True)
)
```

For fine-tuning, we will replace the classifier head with a small MLP: * A `nn.Dropout` layer with 0.2 dropout * A `nn.Linear` layer taking the `num_features` input to the `num_hidden` output * A ReLU activation * A final linear layer with the one output for each class.


```
[18]: num_hidden = 100

# TODO: Replace the classifier head
model.classifier = nn.Sequential(
    nn.Dropout(0.2),
    nn.Linear(num_features, num_hidden),
    nn.ReLU(),
    nn.Linear(num_hidden, len(categories))
)
```

Next, we **freeze** the parameters in all the `model.features` layers, so we only retrain the final layer. This will make the training much faster. You can loop over `model.features.parameters()` and set `param.requires_grad = False`.

```
[19]: # TODO: Freeze all the parameters in model.features
for param in model.features.parameters():
    param.requires_grad = False
```

To confirm we set everything correctly, loop over `model.parameters()` and find the total number of parameters that are trainable and total number that are fixed. You should get that only about 129,000 are trainable.

```
[21]: # TODO: Print the total number of trainable and fixed parameters
trainable_params = 0
fixed_params = 0
for param in model.parameters():
    if param.requires_grad:
        trainable_params += param.numel()
    else:
        fixed_params += param.numel()

print(f"Trainable parameters: {trainable_params}")
print(f"Fixed parameters: {fixed_params}")
```

```
Trainable parameters: 129211
Fixed parameters: 2223872
```

1.5 Loading the model to a GPU

For training image models, it greatly helps to use a GPU.

* Use the `torch.cuda.is_available()` to see if a GPU is available.

* If so, print the number of GPUs with `torch.cuda.device_count()` * Also, print the GPU name with `torch.cudu.get_device_name` and `torch.cuda.get_current_device()`.

```
[ ]: # TODO: See if there is a GPU and what type of GPU
torch.cuda.is_available()
torch.cuda.device_count()
torch.cuda.get_device_name(torch.cuda.current_device())
```

```
# not available on M2 mac
```

```
-----
AssertionError                                Traceback (most recent call last)
Cell In[22], line 4
      2 torch.cuda.is_available()
      3 torch.cuda.device_count()
----> 4 torch.cuda.get_device_name(torch.cuda.current_device())

File ~/Library/CloudStorage/GoogleDrive-wdn2012@nyu.edu/My Drive/F2025/introml/
↳introml/lib/python3.12/site-packages/torch/cuda/__init__.py:1069, in
↳current_device()
    1067 def current_device() -> int:
    1068     r"""Return the index of a currently selected device."""
-> 1069     _lazy_init()
    1070     return torch._C._cuda_getDevice()

File ~/Library/CloudStorage/GoogleDrive-wdn2012@nyu.edu/My Drive/F2025/introml/
↳introml/lib/python3.12/site-packages/torch/cuda/__init__.py:403, in
↳_lazy_init()
    398     raise RuntimeError(
    399         "Cannot re-initialize CUDA in forked subprocess. To use CUDA
↳with "
    400         "multiprocessing, you must use the 'spawn' start method"
    401     )
    402 if not hasattr(torch._C, "_cuda_getDeviceCount"):
-> 403     raise AssertionError("Torch not compiled with CUDA enabled")
    404 if _cudart is None:
    405     raise AssertionError(
    406         "libcudart functions unavailable. It looks like you have a
↳broken build?"
    407     )

AssertionError: Torch not compiled with CUDA enabled
```

Finally, we move the model to the GPU with the following command.

```
[24]: device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
print(f"Using device: {device}")
```

Using device: cpu

1.6 Training the model

We are now ready to train our model. First we select the loss function criterion and optimizer.

```
[25]: # TODO: Set loss function and optimizer
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=1e-4)
```

You can now train the model by completing the following code. With a T4 GPU, each epoch should complete in a few minutes. You should be able to get about 85% accuracy. You can higher accuracy with a larger model, larger classifier head, and more time. But, I want you to just understand the basic ideas.

```
[28]: from tqdm import tqdm

nepochs = 5
for epoch in range(nepochs): # Adjust epochs as needed
    model.train()
    running_loss = 0.0

    loop = tqdm(train_loader, desc=f"Epoch {epoch+1}", leave=True)
    for images, labels in loop:

        # TODO: Move the images to the GPU
        images = images.to(device)
        labels = labels.to(device)

        # TODO: Perform the back-prop on the data
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        # Update the running loss and progress bar
        running_loss += loss.item()
        loop.set_postfix(loss=loss.item())

    print(f"Epoch {epoch+1}, Training Loss: {running_loss:.4f}")

    # Evaluation after each epoch
    model.eval()
    correct = 0
    total = 0

    with torch.no_grad():
        loop = tqdm(test_loader, desc="Evaluating", leave=True)

        for images, labels in loop:
```

```

images = images.to(device)
labels = labels.to(device)

outputs = model(images)

correct += (outputs.argmax(dim=1) == labels).sum().item()
total += labels.size(0)

# Update postfix with current accuracy
accuracy = 100 * correct / total
loop.set_postfix({'Accuracy': f'{accuracy:.2f}%'})

accuracy = 100 * correct / total
print(f"Epoch {epoch+1}, Test Accuracy: {accuracy:.2f}%\n")

```

Epoch 1: 100%| | 412/412 [05:02<00:00, 1.36it/s, loss=1.22]

Epoch 1, Training Loss: 304.9141

Evaluating: 100%| | 155/155 [01:48<00:00, 1.43it/s, Accuracy=84.07%]

Epoch 1, Test Accuracy: 84.07%

Epoch 2: 100%| | 412/412 [05:08<00:00, 1.34it/s, loss=1.46]

Epoch 2, Training Loss: 252.3095

Evaluating: 100%| | 155/155 [01:48<00:00, 1.42it/s, Accuracy=86.25%]

Epoch 2, Test Accuracy: 86.25%

Epoch 3: 100%| | 412/412 [04:55<00:00, 1.39it/s, loss=1.24]

Epoch 3, Training Loss: 225.1821

Evaluating: 100%| | 155/155 [01:43<00:00, 1.49it/s, Accuracy=88.11%]

Epoch 3, Test Accuracy: 88.11%

Epoch 4: 100%| | 412/412 [14:49<00:00, 2.16s/it, loss=1.48]

Epoch 4, Training Loss: 211.4298

Evaluating: 100%| | 155/155 [01:47<00:00, 1.44it/s, Accuracy=88.52%]

Epoch 4, Test Accuracy: 88.52%

Epoch 5: 100%| | 412/412 [04:54<00:00, 1.40it/s, loss=0.461]

Epoch 5, Training Loss: 194.7109

Evaluating: 100%| | 155/155 [01:44<00:00, 1.48it/s, Accuracy=88.96%]

Epoch 5, Test Accuracy: 88.96%

```
[29]: # Save the model
torch.save(model.state_dict(), "mobilenetv2_weights.pth")
```

1.7 Evaluating the model

Let's conclude by evaluating the model. First we reset the model architecture and then load the parameters.

```
[30]: # Set parameters
num_hidden = 100

# Load base model
model = models.mobilenet_v2()

# TODO: Rebuild the classifier head
model.classifier = nn.Sequential(
    nn.Dropout(0.2),
    nn.Linear(num_features, num_hidden),
    nn.ReLU(),
    nn.Linear(num_hidden, len(categories))
)

# Load weights
model.load_state_dict(torch.load("mobilenetv2_weights.pth"))
model.to(device)
model.eval();
```

Now run the model and create a confusion matrix. * Evaluate the model on the `test_loader` dataset * Add a `tqdm` loop to display the progress as it evaluates the data * Create a confusion matrix and display the confusion matrix with `ConfusionMatrixDisplay` function.

```
[39]: from tqdm import tqdm
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

all_preds = []
all_labels = []
with torch.no_grad():
    loop = tqdm(test_loader, desc="Evaluating")
    for images, labels in loop:
        images = images.to(device)
        labels = labels.to(device)
        outputs = model(images)
```

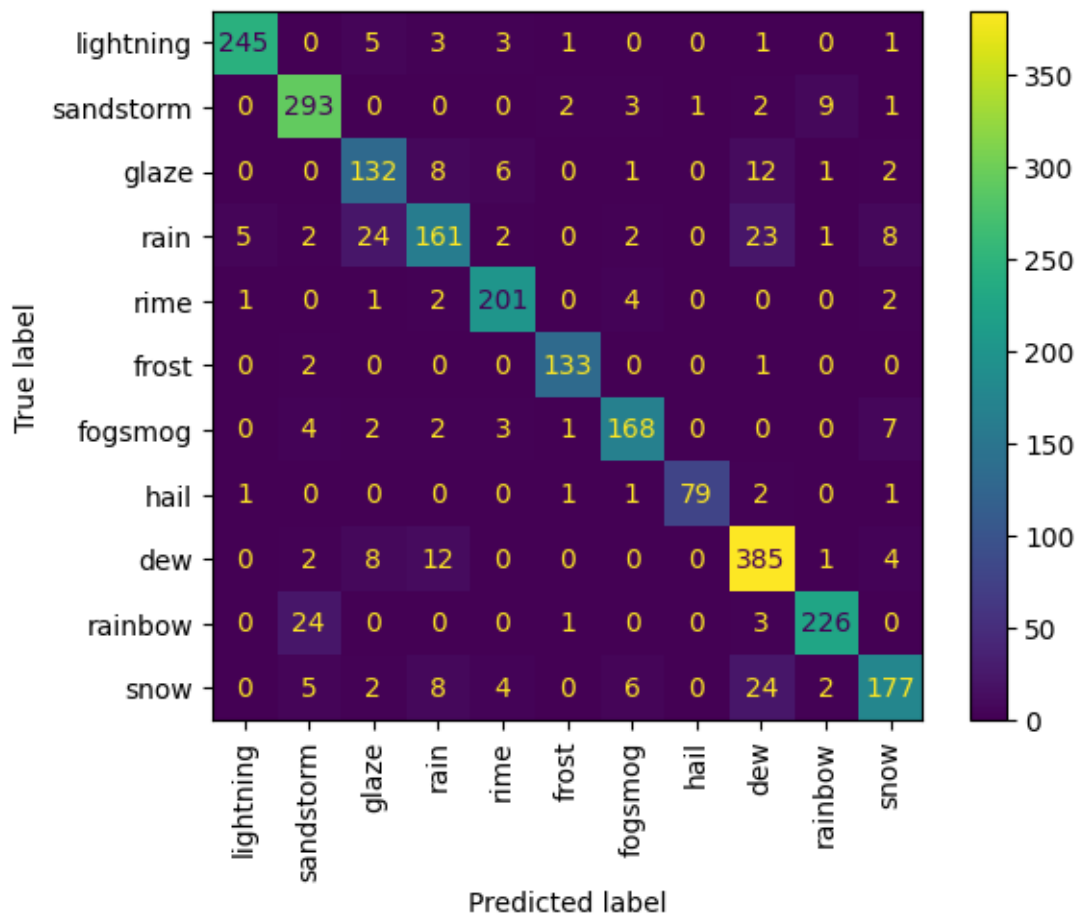
```

_, predicted = outputs.max(1)
all_preds.extend(predicted.cpu().numpy())
all_labels.extend(labels.cpu().numpy())

cm = confusion_matrix(all_labels, all_preds)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=categories)
disp.plot(xticks_rotation='vertical')
plt.show()

```

Evaluating: 100% | 155/155 [01:51<00:00, 1.39it/s]



Finally, print the top k=10 confusion matrix pairs with the highest percent errors. Your final list should be something like: ~~~ Top 10 pairs with the highest error: snow -> glaze: error=0.08750 frost -> snow: error=0.07586 glaze -> hail: error=0.07563 rain -> lightning: error=0.07512 hail -> glaze: error=0.06250 glaze -> snow: error=0.05517 frost -> sandstorm: error=0.05303 snow -> frost: error=0.05128 sandstorm -> frost: error=0.04487 hail -> snow: error=0.04138 ~~~

```
[40]: error_pairs = [(cm[i,j] / cm[i,:].sum(), i, j) for i in range(len(categories))
    ↪ for j in range(len(categories)) if i != j and cm[i,:].sum() > 0]
error_pairs.sort(reverse=True)
print("Top 10 pairs with the highest error:")
for error_rate, i, j in error_pairs[:10]:
    print(f"{categories[i]:>12} -> {categories[j]:>12}: error={error_rate:.
    ↪5f}")
```

Top 10 pairs with the highest error:

```
    snow ->          dew: error=0.10526
    rain ->         glaze: error=0.10526
    rain ->          dew: error=0.10088
rainbow ->    sandstorm: error=0.09449
    glaze ->         dew: error=0.07407
    glaze ->         rain: error=0.04938
fogsmog ->      snow: error=0.03743
    glaze ->         rime: error=0.03704
    snow ->         rain: error=0.03509
    rain ->         snow: error=0.03509
```

[]:

[]: